

and accurate measurement of run time, which is agnostic of external factors that disrupt code execution. Such an operation is often referred to as **micro-benchmarking** which is often used for system level code optimizations and performance tuning.

One might argue against this, in the favor of using the system or the clock time for execution measurement. We could just save the clock time before and after the code execution and the difference would yield the execution time. But such a method will not be accurate nor consistent since there are a number of factors that affect this calculation. We could have different background tasks running momentarily that might disrupt the program execution, hence logging a longer time on the clock.

Even if we are able to get a consistent execution time, the time modules are system dependent and provide varied levels of granularity depending on the operating system we are running, all of which has to be handled separately in our timer implementation. Instead of dealing with these complexities which make our measurements error-prone, **timeit** module comes in handy for such use cases.

The **timeit** module is shipped in two flavors, a **command line** interface and a **python interface** that can be used interchangeably in most of the scenarios.

The **python interface** provides with a convenience method, **timeit.timeit()**; which can be used to perform timing operations. It builds a **Timer** instance and calls the **timeit()** method of the created instance. The method resembles the following signature:

```
timeit.timeit(stmt='pass',setup='pass',timer=<timerFunction>,number=N)
```

stmt is the code block that we are trying to time.

setup is the initial set of statements that is executed once before we start the timer.

timer is a platform dependent timer function pass to the class. It usually has a sensible default value so we need not worry about it.

number is the count of executions to be performed by stmt

#executes only once before test statement

```
test_setup="import math"
```

```
# code snippet under test
```

```
test_stmt = '''
```

```
def example():
```